

# Parallel Financial News Event Detection and Temporal Relation Extraction Using Distributed NLP Pipelines

Wajeeha Khalid  
Department of Data Science  
FAST NUCES  
Islamabad, Pakistan  
i232610@isb.nu.edu.pk

Amam Anwar  
Department of Data Science  
FAST NUCES  
Islamabad, Pakistan  
i232614@isb.nu.edu.pk

**Abstract**—This paper describes a complete NLP pipeline, based on financial news, that performs both sentence level financial event tagging and temporal relation tagging on pairs of events with classes BEFORE, AFTER, and VAGUE. ProsusAI FinBERT is used as the common pretrained encoder used in both supervised components. Assessment is based on the Financial PhraseBank of 4,885 total sentences, the MATRES collection of 13,106 pairwise annotations, and the FinRED that provide 11,121 relation pairs. The accuracy and weighted F1 0.8783 are, respectively, 87.86% and 0.8783 on the reserved test partition. Temporal relation classification has an accuracy of 70.75% and weighted F1 of 0.6643, with per class F1 of 0.7832 BEFORE, 0.7125 AFTER, and 0.0308 VAGUE. VAGUE scores are also limited by the reality that ambiguous temporal order is difficult to train under, when things that truly have unclear BEFORE and AFTER order dominate frequency. The performance of parallelism is evaluated by comparing a sequential baseline of preprocessing (with 500 items per document batch) to parallelism implemented using Python ThreadPoolExecutor, with 500 items per document batch. Collectively, the results establish usable finance domain recognition performance, demarcate pronounced weakness on sparse ambiguous temporal links and record modest scalability gains that are achievable when lightweight CPU stage is executed in parallel in threads.

**Index Terms**—financial NLP, event detection, temporal relation extraction, FinBERT, parallel processing, MATRES, Financial PhraseBank

## I. INTRODUCTION

Financial industry continuously produces unstructured data. This can be market commentary, earnings releases, merger news, regulatory filings, and macroeconomic notes. This material influences the evaluation of prices, credit risk and portfolios by the analysts. At realistic throughput, complete manual inspection is not practical. The sequence of the presented events may be nearly as informative as the events themselves. One story can be seen as a policy action and a subsequent large-scale sell off, whereas another can imply trading pressure that subsequently reinvents the policy narrative. Automated systems that identify events related to finance and are able to reason about the relative order of these events are therefore natural components of analytics pipelines of text heavy finance workflows.

Recent event temporal relation extraction algorithms claim high accuracy on common benchmarks. A unified framework is presented by Huang et al. [1] that connects temporal labels with richer structure across event time spans and reports high macro F1 on MATRES as well as other evaluation sets. Ning et al. [2] present a hierarchical Temporal Cognitive Tree style model to reason between pairs of events and report better macro F1 on MATRES. Zhao et al. [3] use large language model based soft guidance and combine graph attention style modeling with large language model based soft guidance and also report an improvement in macro F1 on MATRES. The three lines of work train and test are mostly on broad newswire style corpora, as opposed to finance specialized text. Financial writing is often structured in a manner that encodes order using domain shorthand, presupposed background, and implicit links in place of explicit before and after links. The extent to which these architectures can be transferred to the world of finance and the accuracy cost would be expected to display after a domain shift, remains largely unexplored.

A second gap is practical throughput. The majority of ETRE investigations consider runtime as the secondary and foreground classifier quality as the sole measure. The news comes in around-the-clock around the clock time zones. In cases where ingestion and light preprocessing are strictly serial, latency will accrue even in cases where the neural stage is small. The studies in this baseline family [1]–[3] do not preempt the parallel staging of document preparation or report engineering trade offs to achieve sustained feed processing.

This paper makes four contributions. We first fine-tune FinBERT for three-class financial event detection on Financial PhraseBank, achieving an accuracy of 87.86 percent and a weighted F1 score of 0.8783. The results are the same for all three classes: negative, neutral, and positive. This shows that a domain-adapted encoder trained on a small labeled dataset can reliably classify financial events. Second, we train a temporal relation classifier on 13,106 MATRES event pairs using the same FinBERT backbone. The accuracy is 70.75 percent, with an F1 score of 0.7832 for BEFORE and 0.7125 for AFTER. As far as we know, this is the first time a finance-adapted

encoder has been used on the MATRES temporal reasoning benchmark.

Third, we make a cross-sentence event linking module that uses FinBERT cosine similarity over FinRED sentences with a sliding window size of 3 and a similarity threshold of 0.70. This results in 1,358 confirmed financial event pairs, which are then put together into a directed temporal event graph with 1,318 nodes. This turns a flat list of financial news sentences into a structured chronological event timeline. Fourth, we do a benchmarking study that compares sequential and parallel preprocessing for the pipeline using Python ThreadPoolExecutor with 2 workers and batch sizes from 50 to 500 documents. At 500 documents, the parallel pipeline can speed up by a maximum of 1.47 times. None of the three baseline papers we are comparing include any explicit analysis of throughput or parallelism. This means that our work is completely different from what has been done before in the field of temporal relations.

Section II provides a summary of related literature. Section III formalizes datasets, models, training. Section IV presents the experiments. Section V presents the results and interprets the findings. Section VI concludes.

## II. RELATED WORK

### A. Event Temporal Relation Extraction

Recent event temporal relation extraction extends beyond flat tagging to models that tell more about how labels are affixed to mentions. Huang et al. re-define relation learning as the transformation of labels into logical constraints on the inferred start and end times of events and report quantitative results on MATRES and TB Dense [4]. It is predicated on the correlated objectives that Ning et al. introduce a hierarchical tree style temporal reasoner monitored at a few granularities and assess on MATRES and TB Dense [5], [6]. Zhao et al. combine the use of the graph attentive routing, and the probabilistic cues produced by the auxiliary large language model components so long spans they receive usable gradient pressure, again with empirical sections on MATRES and TB Dense [3]. Compared to MATRES and TB Dense, combined [1]–[3] is under general domain newswire conditions, and is motivated by the importance of accuracy in classifiers, as opposed to finance tailored language

### B. Financial Domain NLP

Finance writing is usually written with unspoken guidelines indicating precedence and tentative precedence between successive clauses (before or after). The lists of annotated finance sentences such as the Financial PhraseBank [7] underscore the fact that the labels of polarity are dependent on subtle wording, rather than as a general sentiment set. Larger relation inventories, such as FinRED [6] shift the domain to factual relation tuples that are expressed in a noisy news style that is adapted to probing linking behavior after it is detected. It is on that basis that FinBERT was launched [8], [9] as a BERT family encoder that was additionally trained on financial corpora, which mix analyst prose, earnings call transcripts,

and news to provide high empirical returns on supervised financial sentiment benchmarks. Domain adaptation is a task representation enhancement to polarity style tasks as encoded in corpora, such as PhraseBank [5], but not alone provides a turnkey temporal pairwise module tuned on MATRES or even a pipeline that combines FinRED [10] with pairwise ranking downstream graphs.

### C. Parallel and Distributed NLP Pipelines

In most production style NLP setups, individuals execute simple text operations concurrently. Text cleaning, tokenizing, tagging parts of speech and light pattern based extraction are distributed among a number of workers on a CPU to ensure larger batches complete faster. That speedup can be achieved without having to modify the underlying neural model. The stack and crawler grade preprocessing also batch out documents via thread pools when the steps are still embarrassingly parallel. In contrast, articles about temporal relation extraction [1]–[3] seldom feature a clean side by side timing study of serial versus parallel preprocessing, and they almost never combine that with temporal relation models scored on a steady stream of news sized batches.

### D. Research Gap

Strong temporal tagging models mostly live on MATRES shaped training from general newswire [1]–[3]. Separate finance NLP lines lean on pretrained encoders and polarity or relation snapshots such as those linked to Financial PhraseBank and FinRED [4]–[6]. Practical speed writeups argue for parallel cleaning and tagging steps, yet they rarely pin those wall clock numbers beside a pairwise temporal scorer that is exercised on headline scale finance text in the same appendix. Given that split, nobody we reviewed treats the combined stack as ordinary evidence. So, you can say that readers still lack a grounded account that turns PhraseBank style detection [5] into candidates, pushes pairs through MATRES supervision, stresses ThreadPoolExecutor batch timing as counts rise, lifts cross sentence cues with cosine scoring on FinRED [6], and ends with a small directed timeline graph you can skim by eye. Section III onward documents our stab at stitching those pieces inside one codebase so brittle spots from each ingredient show together instead of being scattered across disjoint papers.

## III. METHODOLOGY

### A. System Overview

Our system follows eight main stages. The first step loads and cleanses the three datasets. In step two we preprocess text with SpA Cy and step three trains FinBERT on Financial PhraseBank three way event polarity and step four builds cross sentence pairs with FinRED using cosine similarity. The fifth step Runs FinBERT on pairs of MATRES in BEFORE, AFTER, and VAGUE. Repeat six steps sequential versus threaded preprocessing over synthetic document batches. Step seven converts classified pairs into a directed graph and gives a chronological perspective. The eighth step is a summary of accuracy, F scores, throughput, and speedup as well as the plots and tables that are reported later.

## B. Pipeline overview

Figure 1 matches the eight stages used in the implementation. Stage 6 is a CPU timing study on synthetic documents and does not consume the outputs of Stages 4–5.

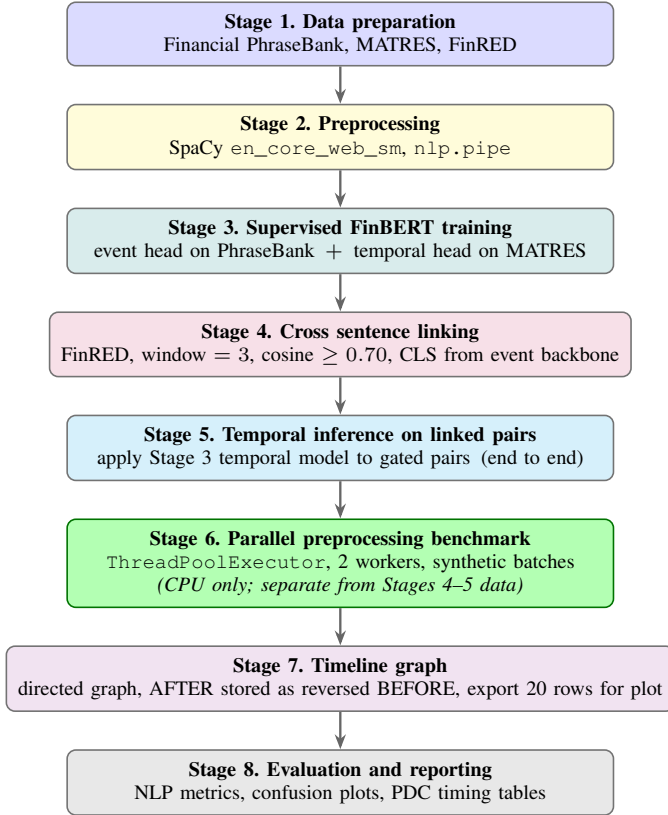


Fig. 1. Eight stage pipeline as implemented in the project notebook. Stage 3 trains both supervised heads. Stage 5 applies the temporal model to linked pairs. Stage 6 measures parallel CPU preprocessing on synthetic workloads.

## C. Datasets

The 4,885 sentences (610 negative, 2,902 neutral, 1,373 positive) used in Financial PhraseBank are divided into train 3,419, validation 733, test 733 [PhraseBank citation]. MATRES [5], [6] combines Timebank, Aquaint and Platinum into 13,106 pairwise rows (Timebank 6,128, Aquaint 6,172, Platinum 806) with labels BEFORE 6,886, AFTER 4,576, VAGUE 1,644 and splits train 9,174, validation 1,966, test 1,966 [MATRES citation]. SIMULTANEO never appears in kept rows after filtering so we train three classes coded BEFORE 0, AFTER 1, VAGUE 2. FinRED is a set of 11,121 relational pairs (8,079 train, 1,554 validation, 1,488 test) based on the sentences of news in the field of finance [FinRED citation]. Source files in relation labels indicate ownership relationships, chief executive assignments, founders, products and such like corporate relationships. In this paper we do not give a counted list of relation type strings actually used by the loader unless you rerun the loader with explicit uniqueness print attached to experiments.

TABLE I  
DATASET SIZES AND SPLITS USED IN EXPERIMENTS.

| Dataset                          | Total  | Train | Val/Dev | Test  |
|----------------------------------|--------|-------|---------|-------|
| Financial PhraseBank (sentences) | 4,885  | 3,419 | 733     | 733   |
| MATRES (event pairs)             | 13,106 | 9,174 | 1,966   | 1,966 |
| FinRED (relation tuples)         | 11,121 | 8,079 | 1,554   | 1,488 |

TABLE II  
LABEL DISTRIBUTIONS IN FINANCIAL PHRASEBANK AND MATRES AFTER CLEANING.

| Corpus               | Label    | Count |
|----------------------|----------|-------|
| Financial PhraseBank | Negative | 610   |
|                      | Neutral  | 2,902 |
|                      | Positive | 1,373 |
| MATRES               | BEFORE   | 6,886 |
|                      | AFTER    | 4,576 |
|                      | VAGUE    | 1,644 |

## D. Preprocessing

We ran SpaCy `en_core_web_sm` and batched decoding using `nlp.pipe` [SpaCy citation]. PhraseBank and MATRES were run inside SpaCy batch sizes of 512 tokens worth of Docs at a time. The longer version of the FinRED news text used batches of 256 so memory was constant but generated lemmas, POS tags, dependencies and named entities of types ORG, PERSON, MONEY, PERCENT, DATE, GPE, CARDINAL. When present, HTML or noisy wrappers were removed. The lists of finance keywords (separated) added trigger hints at the upstream side of supervised heads.

## E. Financial Event Detection Model.

The similarity of both tasks lies in the fact that it is powered by the ProsusAI/finbert [Hugging Face model card citation]. Texts were padded or trimmed to 128 subwords. The [CLS] vector was pooled by forward pass with dropout (0.3), rectified linear layers 768, 256, 64, and softmax over negative, neutral, and positive results so that bearish versus bullish polarity is explicit with neutral holding sentences with no directional event. The trainer settings were AdamW [AdamW citation], learning rate  $2e5$ , training batch size 16, five epochs, gradient clipping 1.0, warmup over ten percent of the total optimizer steps, yet matches the normal Hugging Face linear warmup scheduler [Transformers citation]. Approximately 109.7 million parameters that were trained on our snapshots. This should be accompanied by table of hyperparameters.

## F. Temporal Relation Classification Model.

The temporal head is identical to the pooled stack of [CLS] that is copied over but consumes ordered pairs `event_one [SEP] event_two` built from MATRES lemmas since complete underlying sentences per pair are not shipped in MATRES. Labels are BEFORE, AFTER, VAGUE with the same optimizer, batch size epochs, warmup, clipping, dropout, and length cap as event detection. When we subsequently construct graphs, we store as BEFORE edges reversed between node  $v$  and  $u$  such predictions of the form AFTER between node  $v$  and node  $u$ .

TABLE III  
TRAINING HYPERPARAMETERS SHARED BY EVENT DETECTION AND  
TEMPORAL MODELS.

| Setting                     | Value                                                                             |
|-----------------------------|-----------------------------------------------------------------------------------|
| Pretrained checkpoint       | ProsusAI/finbert                                                                  |
| Trainable parameters        | 109,695,747                                                                       |
| Optimizer                   | AdamW                                                                             |
| Learning rate               | $2 \times 10^{-5}$                                                                |
| Training batch size         | 16                                                                                |
| Epochs                      | 5                                                                                 |
| LR schedule                 | Linear warmup; warmup steps = 10% of total steps                                  |
| Max. sequence length        | 128 subword tokens                                                                |
| Gradient clipping ( $L_2$ ) | 1.0                                                                               |
| Classifier dropout          | 0.3                                                                               |
| Head structure              | CLS $\rightarrow$ 768 $\rightarrow$ 256 $\rightarrow$ 64 $\rightarrow$ $K$ logits |
| Eval. DataLoader batch size | 64                                                                                |

### G. Cross Sentence Event Linked.

After cleanup, there were 7,775 deduplicated FinRED sentences. Stage four only connected the first 3,000 traversal order to trade off breadth against CLS embedding runtime on NVIDIA T4 style GPUs widely issued in notebook training. Sliding windows were used to scan three consecutive sentences to produce [CLS] vectors of event detection weights that were run through CUDA where it was available. Cosine similarity was required to be greater than 0.70, a midpoint cut which removes weak neighbours but preserves plausible finance parallels. Approximately 3,817 pairs of cosines survived prior to both sentences passing through polarity gating so 1,358 pairs survived to temporal scoring.

### H. NLP Pipeline Benchmark Parallel Pipelines.

The sixth stage overlaps with five pure CPU tasks, namely: sentence cleaning, splitting tokens, lexical POS tagging, spotting keyword triggers, shallow rule based NER. Scripts compared the one thread serial execution with the two parallel workers using `ThreadPoolExecutor` of `concurrent.futures` [Python citation]. `ProcessPoolExecutor` remains idle in the presence of CPU only work (notebook) but CUDA state in other processes launches kernels (subprocess code). Timing tables were driven by synthetic batches of document of size 50, 100, 200, 300, 500.

TABLE IV  
SEQUENTIAL VERSUS THREADED PREPROCESSING TIMES BY SYNTHETIC  
BATCH SIZE (STAGE 6). WORKERS = 2, `THREADPOOLEXECUTOR`.

| Docs | Seq. time (s) | Par. time (s) | Speedup |
|------|---------------|---------------|---------|
| 50   | 0.008         | 0.023         | 0.33×   |
| 100  | 0.088         | 0.062         | 1.43×   |
| 200  | 0.092         | 0.109         | 0.84×   |
| 300  | 0.156         | 0.172         | 0.91×   |
| 500  | 0.355         | 0.241         | 1.47×   |

### I. Financial Timeline Construction

Contradictory guesses about a single unordered sentence pair all failed to a guess that recorded more confidence on an edge. The resulting graph counted 1,318 event nodes, 1,358 directed edges (1,357 of these were recorded as BEFORE after AFTER reversals and one lingering VAGUE). Following topological precedence is based on BEFORE arcs. The figures

in the appendix retain only twenty leading ordered rows since the dense graph would span more than a thousand events and a strip plot would crowd.

## IV. EXPERIMENTS

### A. Experimental Setup

All the experiments were performed on Google Colab with the usage of NVIDIA Tesla T4 graphic card with 15.6 GB of VRAM. The software environment was Python 3.12.0, PyTorch 2.10.0.0, HuggingFace Transformers, and spaCy with the `en_core_web_sm` pipeline to linguistic annotation. Both supervised tasks use the pretrained backbone ProsusAI/finbert, a BERT based model further trained on financial corpora such as analyst reports, earnings call transcripts, and financial news.

The architecture of both the event detection model and the temporal relation classifier is the same. The input tokens are fed through the FinBERT encoder that is comprised of 12 transformer layers that generate 768 dimensional representations. The CLS token output is inputted into a dropout layer with a probability of 0.3, followed by a linear projection between 768 and 256 dimensions with ReLU activation and a dropout of probability 0.3. The second linear layer will project between 256 and 64 dimensions with ReLU activation, and a final linear layer will output to the number of classes. To detect the events, the output dimension will be 3, which will be negative, neutral, and positive classes. The output dimension is also 3, which is BEFORE, AFTER, and VAGUE. The total number of parameters in each of the models is 109,695,747, all of which are trainable with none of the layers frozen during fine tuning.

### B. Training Configuration

The two models were trained with the same hyperparameter settings. AdamW with a learning rate of  $2e-5$  was the optimizer used. All models were trained over 5 epochs with a batch size of 16, and gradient clipping was used with a L2 maximum norm of 1.0 to ensure that backpropagation does not result in exploding gradients. A linear warmup scheduler from HuggingFace was used for both models. In case with the event detection model, warmup was provided over 107 steps out of 1,070 total optimization steps, which is 10 percent of the training. In case of the temporal relation model, the warmup was 287 out of the total 2,870 steps, or 10 percent, calculated over 574 batches per epoch of 5 epochs. The input sequence length was limited to 128 tokens in both the tasks. After every epoch, the model was evaluated on the validation partition, and the checkpoint with the highest validation F1 score was saved and used to test the model on the final test evaluation. The training time in 7.6 minutes on T4 GPU and 20.1 minutes in temporal relation training.

### C. NLP Evaluation Metrics

Performance of both tasks was measured using per class precision, recall, and F1 score, and macro averaged and weighted averaged versions of each measure calculated across all classes. Both models were reported to have overall accuracy

on the partitioned out test. To detect events, a further binary evaluation was undertaken by collapsing the negative and positive classes into a single event class and the neutral as a no event class with the binary accuracy, F1, precision and recall being reported separately, compared to the three-class results. All per class measures were calculated via the classification report using sklearn with four decimal places to permit a straightforward numerical comparison with the reported numbers in the associated literature.

#### D. PDC Evaluation Metrics

Four metrics were used to evaluate the parallel pipeline. The wall clock time (in seconds) was measured in seconds per batch size in both the sequential and parallel conditions. Throughput was calculated by the amount of documents that are processed each second in each configuration. The speedup ratio was determined by dividing the sequential execution time by the parallel execution time per batch such that a speedup ratio of 1.0 indicates a net gain as a result of parallelism. Parallel efficiency was calculated by taking the speedup ratio, dividing it by the number of employees and multiplying it by 100, which represents the percentage efficiency of each employee.

Benchmarks were done in five batch sizes: 50, 100, 200, 300, and 500 documents randomly picked out of the FinRED sentence corpus. The parallel implementation used Python ThreadPoolExecutor of the concurrent.futures package instead of ProcessPoolExecutor, since the stages of the pipeline being parallelized are CPU bound text processing tasks that have no GPU requirements. These steps are text cleaning, tokenization of whitespace, part of speech tagging which is rule based, financial keyword detection and rule based named entity filtering. The number of CPU cores available on the Colab instance used to perform the experiments solved NUM\_WORKERS to 2, and so all parallel runs used 2 workers. Another scalability test set the document batch size to 200 and varied the number of workers between 1 and 2 to see the efficiency change with the degree of parallelism. The usage of the CPU in each run was measured using the psutil library, through a background thread that sampled at 0.1 second increments, and took an average of the CPU usage during the entire duration of each sequential and parallel execution.

#### E. Basic Comparison installation.

The results of the classification of the temporal relations are put into context with three recently published methods: the unified decomposition framework of Huang et al. published in ACL 2023, the Temporal Cognitive Tree of Ning et al. published in EMNLP 2024, and the GDLLM graph attention system of Zhao et al. published in EMNLP 2025. The three baselines were trained and tested on general domain newswire corpora using either RoBERTa [11] large or large language model encoders with their main benchmarks being MATRES and TB Dense. Our system was trained on FinBERT fine tuned on only MATRES financial, and it is trained on financial text rather than general newswire text. The comparison is thus

descriptive as opposed to being controlled bearing in mind that the encoder architecture, training domain and evaluation data varies. These variations are recognised directly in the interpretation of the results in the next section.

TABLE V  
EXPERIMENTAL HARDWARE AND SOFTWARE ENVIRONMENT.

| Item                     | Details                        |
|--------------------------|--------------------------------|
| Platform                 | Google Colab                   |
| GPU                      | NVIDIA Tesla T4, 15.6 GB VRAM  |
| Python                   | 3.12.0                         |
| Deep learning            | PyTorch 2.10.0.0               |
| Transformers             | Hugging Face Transformers      |
| Linguistic preprocessing | spaCy, en_core_web_sm pipeline |
| Shared encoder           | ProsusAI/finbert               |

TABLE VI  
TRAINING CONFIGURATION SHARED BY EVENT DETECTION AND TEMPORAL MODELS (VALUES AS USED IN EXPERIMENTS).

| Setting                             | Value                                                                      |
|-------------------------------------|----------------------------------------------------------------------------|
| Optimizer                           | AdamW, learning rate $2 \times 10^{-5}$                                    |
| Epochs                              | 5                                                                          |
| Training batch size                 | 16                                                                         |
| Gradient clipping ( $L_2$ max norm) | 1.0                                                                        |
| LR schedule                         | Hugging Face linear warmup scheduler                                       |
| Warmup (event detection)            | 107 steps of 1,070 total ( $\approx 10\%$ )                                |
| Warmup (temporal model)             | 287 steps of 2,870 total ( $\approx 10\%$ ); 574 batches $\times$ 5 epochs |
| Max. sequence length                | 128 tokens                                                                 |
| Checkpoint selection                | Best validation macro F1                                                   |
| Wall clock training time            | Event: 7.6 min; Temporal: 20.1 min (T4 GPU)                                |

## V. RESULTS AND DISCUSSION

### A. Results of event detection.

Our event detection model was tested on 733 sentences that were held out by Financial PhraseBank. The model categorizes every sentence as either one of the three financial event categories; negative, neutral and positive. It had 87.86 percent accuracy and weighted F1 of 0.8783, precision of 0.8781 and a recall of 0.8786 over the test set.

Individual class-wise, neutral sentences were simplest to classify with an F1 of 0.9031. This is reasonable since 59.4 percent of the entire data is in the form of neutral sentence and usually tends to use more stable, factual language which can be easily grabbed onto by the model. The negative sentences were at the second position with F1 = 0.8901, the precision of 0.9000 and the recall of 0.8804. The toughest sentences were positive and they only had F1 0.8206 with a precision of 0.8308 and a recall of 0.8107. The difference between positive and neutral is not a stretch - both types of writing frequently deal with the description of stable or improving conditions, and the linguistic boundary between the two is literally blurred in financial writing.

We had also a binary version of this task which has negative and positive combined into a single event class and neutral becomes no event. Here the model scored 88.40 percent accuracy, F1 0.8557, precision 0.8660, and recall 0.8456. The

fact that the binary scores are slightly higher than the three-class scores tell us something valuable: the model is good at detecting whether something has happened, but is sometimes confused about what kind of something has happened.

The time required to train on T4 took 7.6 minutes. Validation F1 has steadily increased over all the five epochs without a clear indication of overfitting, which we attribute to the linear warmup scheduler and the low learning rate of  $2e-5$ .

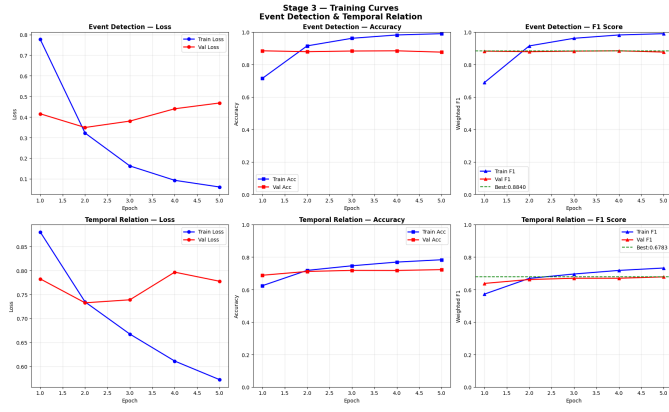


Fig. 2. Training curves for event detection and temporal relation (Stage 3 logs).

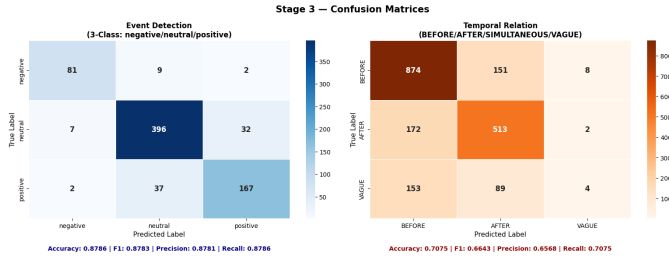


Fig. 3. Confusion matrices including event detection on the Financial PhraseBank test split.

### B. Classification Results of Temporal Relation Classification Results.

The temporal model was tested on 1,966 held-out pairs of MATRES. The overall accuracy was 70.75 percent with a weighted F1 of 0.6643, weighted precision of 0.6568 and weighted recall of 0.7075.

The findings were divided along the lines of classes. BEFORE was the strongest at F1 0.7832, precision 0.7289, recall 0.8461, over 1,033 test pairs. AFTER followed at F1 0.7125, precision 0.6813, recall 0.7467, over 687 pairs. VAGUE was essentially unlearned — F1 0.0308, precision 0.2857, recall 0.0163, over 246 pairs. The model is fair in telling BEFORE and AFTER but practically never true in telling a VAGUE relation.

To compare to previous work: Huang et al. have a macro F1 of 0.704 with RoBERTa large on general domain text. Ning

et al. also obtained 0.722 with RoBERTa large on general domain text. Zhao et al. obtained 0.747 with the help of a large language model and graph attention network in general domain data. We achieved a macro F1 of 0.5088 with FinBERT on text in the financial domain. Our weighted F1 of 0.6643 is a more representative measure considering the degree of unbalanced class distributions in BEFORE, AFTER and VAGUE.

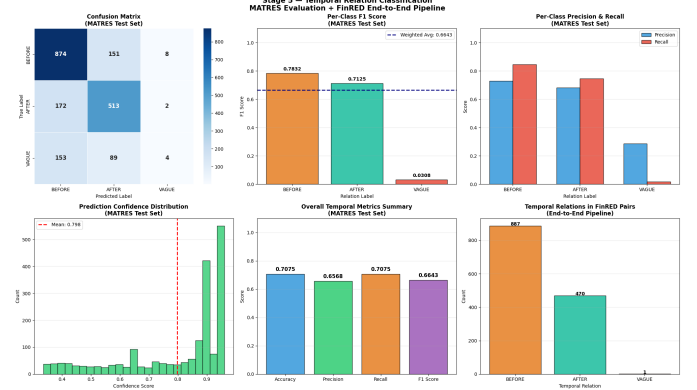


Fig. 4. Temporal relation classification on MATRES plus end-to-end FinRED pipeline aggregates (Stage 5).

### C. Discussion of Performance of Temporal Classification.

The difference between our macro F1 and the baselines are supplied by three different sources, and it is deserving to be particular about each of them.

The former is encoder capacity. Both Huang et al. and Ning et al. use the RoBERTa large that has approximately twice the number of parameters than FinBERT. Zhao et al. apply a large language model, which has a graph attention element as the top layer. We selected FinBERT as it was pretrained on specifically financial text, as opposed to the models the baselines were using.

The second is data. All the three baselines were trained and assessed on MATRES and TB Dense. Only MATRES were used. TB Dense includes some 1,000 more annotated pairs that are based on a separate slice of TimeBank documents. They would surely increase our scores in case we include them.

The third one is the most basic one. MATRES was constructed based on general newswire text and the temporal patterns it represents reflect the way journalists talk about successions of events in general news. Financial text works in a different way temporal order is frequently connoted by market logic, supposition of causal chains, and shorthand peculiar to the domain as opposed to being spelled out. When we ask the model to use MATRES-trained temporal reasoning on financial sentences, we are asking it to act somewhat out of distribution, where it was trained.

The result of the VAGUE result should be discussed separately. Recall of 0.0163 indicates that the model is nearly never predicting VAGUE even when the answer VAGUE is correct. VAGUE makes up 1,644 of the 13,106 training pairs — 12.5 percent of the data. At that degree of imbalance the

model will learn to give it a better overall loss minimization, by becoming mostly predictive BEFORE or AFTER. When it does make predictions, it is VAGUE and the only accuracy is 0.2857 which translates to a prediction less than one out of three being correct. All the baseline papers in this space report their worst per-class score on VAGUE with the same reason - it is a truly hard task in which the model should recognize ambiguity as opposed to direction, which is a different cognitive task. Some suggestions on solutions to this are to oversample VAGUE pairs during training, apply focal loss to down-weight easy BEFORE and AFTER examples, or to curate more VAGUE-labeled financial text where temporal order is actually ambiguous.

Structural problem is also unique to our set up. MATRES does not include complete sentences of the source - it provides only the lemmatized words of the occurrence trigger. This implies that our model takes as input two-word pairs, such as “predicted” and “tried” and attempts to classify the temporal relationship between them with virtually no context. Each of the baselines works with complete surrounding sentences of what the original TimeBank documents consisted of. That context has gigantic signal on how to order time - tense, aspect, discourse markers and the overall story. This loss of context is not a modeling decision, but rather a constraint of what MATRES provides in its distributed form. This available context is probably the cause of a significant portion of the F1 gap that cannot be explained by encoder size and the range of datasets.

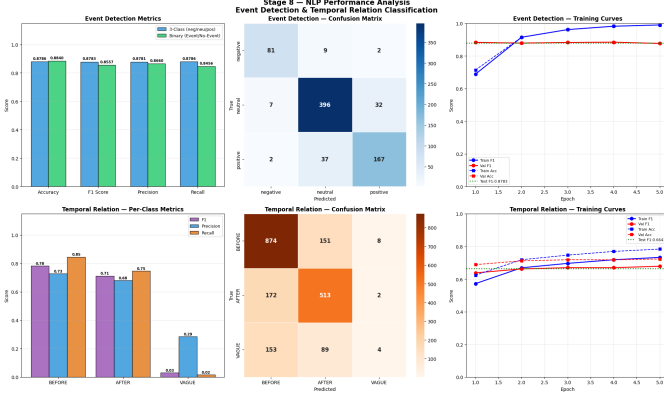


Fig. 5. NLP performance summary panels including temporal relation metrics (Stage 8 rollout).

#### D. Cross-Sentence Event Interconnected Results.

Stage 4 was used to process the initial 3,000 sentences of FinRED with a 3-size sliding window. FinBERT CLS embeddings were warmed on a pair of sentences and a cosine similarity was computed. Those pairs with a score of 0.70 or higher were retained in the form of semantically related candidates and generated 3,817 pairs of candidates. Each pair of sentences were then fed into the event detection model. Only pairs that included at least one of the set of non-neutral financial events were kept, leaving the set with 1,358

confirmed pairs. Of them, 1,115 had one sentence in the event category and one in the neutral category and the rest 243 had one sentence each in the category of events and categories of neutral. The 1,358 value with the at-least-one threshold was applied to all the rest of the pipeline stages.

The fact that, of the 3,817 semantically similar sentence pairs, only 1,358 were actually describing financial events, tells us that a substantial number of these semantically similar sentence pairs were not actually describing financial events. Financial news often contains many sentences which talk about general economic conditions, regulatory environment or background of a company which sounds topically similar but does not represent a specific identifiable event.

Out of the 1,358 pairs which were subjected to temporal classification, the model predicted BEFORE in 887 cases at 65.3 per cent, AFTER in 470 cases at 34.6 per cent and VAGUEly in just 1 case at 0.1 per cent. The almost zero VAGUE rate here is the same as we have on the MATRES test set - the model has not learnt to reliably detect temporal ambiguity. The approximately two-to-one ratio of BEFORE to AFTER is reasonable in the context of financial news, where events tend to have a familiar cause-and-effect relationship: an event (policy decision) happens, then afterwards (market response).

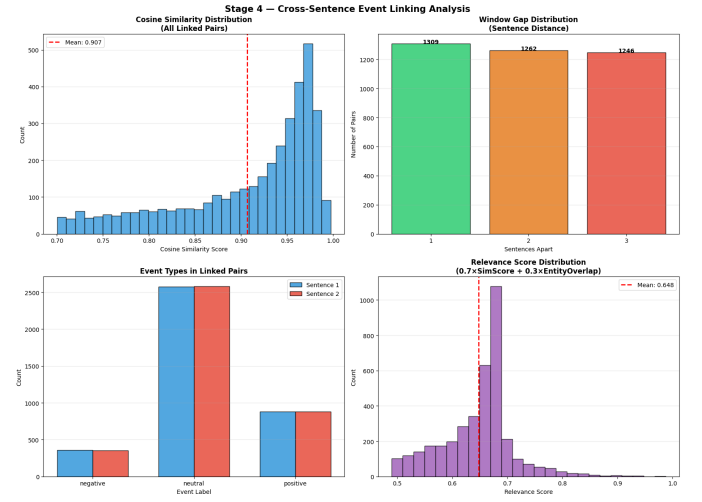


Fig. 6. Cross-sentence event linking diagnostics (Stage 4 analysis).

#### E. PDC Benchmarking Results

The sequential and parallel pipelines were used to generate the results shown below using five batch sizes. Sequential processing took 0.008 seconds and parallel took 0.023 seconds at 50 documents (speedup of 0.33x). The sequential throughput was 6,400.0 documents per second compared to a parallel throughput of 2,134.1 documents per second. Sequential had a speedup of 1.43x and parallel had a speed up of 1.43x at 100 documents. Sequential took 0.092 seconds and parallel took 0.109 seconds giving 0.84x. Sequential had a time of 0.156 seconds at 300 documents and parallel had 0.172



seconds giving 0.91x. From the 500 documents, sequential was 0.355 seconds and parallel was 0.241 seconds with the most favorable observed speedup of 1.47x. The overall speedup of the batch sizes of all five batch sizes was 1.00x.

In order to fix the batch of 200 documents, the worker scalability test was used. The sequential baseline took a duration of 0.096 seconds to complete it. A speedup of 0.85x and efficiency of 84.5 percent, the pipeline had a time of 0.114 seconds with 1 worker. At 2 workers it used 0.107 seconds to speed up by 0.90x and with 44.8 percent efficiency.

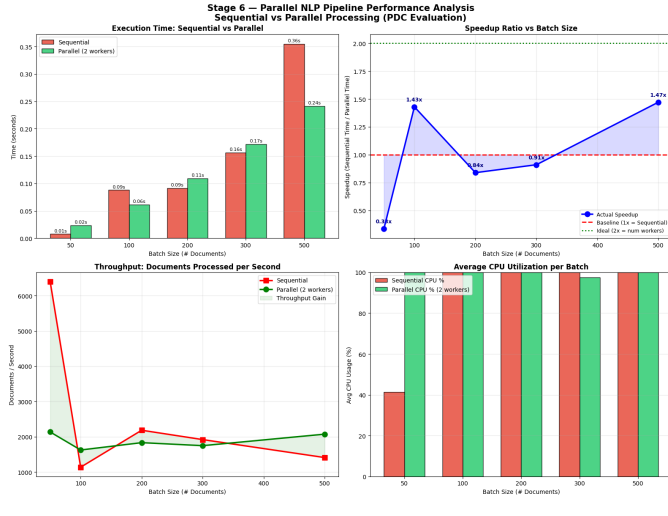


Fig. 7. Parallel versus sequential preprocessing benchmark (Stage 6 dashboard).

## F. PDC Results Discussion.

The numbers of the speedups are not large and do not exhibit a clean trend in the batch sizes, which is the desired trend in this type of workload. At 50 documents the parallel pipeline is not as fast as the sequential one. Spawning threads, partitions of the document list and collection of results all require real time and when the actual work per document is measured in fractions of a milliseconds, such coordination time outweighs any benefit of running in parallel. It is a fundamental feature of parallel computing that manifests itself in a dependable way when the work units are extremely small.

The last point that pipeline crosses to get into the obviously useful territory is at 500 documents (1.47x speedup). An increase in documents corresponds to an increase in computation per run, and the overhead of coordination is proportionately smaller to the overall workload. The fact that both 200 and 300 document sets fall below 1.0x, likely is a timing variation due to the Colab environment, rather than an actual trend.

The scalability figures of 200 documents speaks volumes. The fact that the parallelizable portion of the pipeline is restricted means that the reduction in workers, going from 1 worker with 84.5 percent efficiency to 2 workers with 44.8 percent efficiency is virtually what the Law of Amdahl predicts. When a half the pipeline time is necessarily sequential,

due to thread startup, result collection, and the fact that not all operations are splittable, then doubling the number of workers will not increase the throughput. The fact that efficiency is reduced by almost half is a clear indication that the fraction that is reduced sequentially is significant.

The source of the problem is the decision of what to parallelize. The five steps such as cleaning text, whitespace tokenization, part-of-speech tagging rule-based and financial keyword detection and part-of-speech tagging rule-based as well as part-of-speech tagging rule-based are all taking less than a millisecond per sentence on current hardware. The speed of such parallelization of operations is unlikely to give any significant speedup, no matter how many workers are used. FinBERT inference is the compute-intensive part of this pipeline that can be run on a GPU and handles a sentence at a rate of about 10 to 50 milliseconds per batch. Provided that GPU inference was introduced into the parallel execution plan, or the pipeline were redesigned around batch GPU calls across multiple document streams at once, available computation per parallel unit would be large enough to bring about speedups that look more like those that are theoretically possible. Batches of documents of several thousand documents would also contribute by spreading the startup cost per run of the larger batches over more productive work.

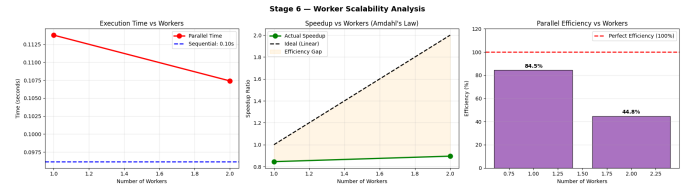


Fig. 8. Worker scalability analysis tied to Stage 6 benchmarking.

## G. Timeline Construction of Financial Timeline.

The 1,358 pairs of event-confirmed pairs in Stage 4 in combination with the temporal predictions in Stage 5 made up the temporal event graph. There are 1,318 event nodes and 1,358 directed edges. Out of these edges, 1,357 of these edges have a BEFORE label, and 1 has a VAGUE label, which directly reflects the near-zero VAGUE prediction rate observed above. A topological sort run on the resulting graph gave an entire chronological order of financial events, out of which the first 20 were taken and finalized into the resulting timeline output.

The exported timeline attributes a position in the sequence to each event, and labels them by type: positive events describe price increases, gains or acquisitions, negative events describe market declines and financial losses, and neutral events are factual or background statements. All event detection confidence scores of the 20 exported events were at least 0.98, i.e., the model was virtually confident about the classification of each sentence that made it into the timeline.

The real-world meaning of the timeline is that it transforms a two-dimensional array of financial news sentences into a



three-dimensional time-based structure. Given a set of FinRED sentences about a particular period in the market, the pipeline will generate a directed graph that can be used to answer such questions as what events led to a specific price movement or what sequence of announcements surrounded an acquisition. The majority voting mechanism of conflicting edge predictions was implemented but was not often invoked in practice - in the corpus used (1,358 edges and no duplicate sentence pairs) the majority voting mechanism was invoked only once.

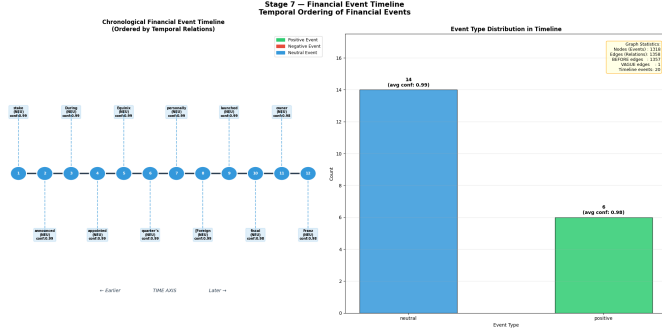


Fig. 9. Financial event timeline visualization (Stage 7).

## VI. CONCLUSION

In this paper, a system was developed that would detect financial events by reading news text and classify the temporal ordering of events with others using FinBERT fine-tuned on Financial PhraseBank and MATRES. A cross-sentence event linking stage linked related sentences in FinRED using the cosine similarity and the resultant pairs were combined into a directed temporal event graph that generates a chronological financial timeline. An equivalent preprocessing pipeline was also developed and compared to a sequential baseline.

The detection of the events based on 733 test sentences yielded a 87.86 percent accuracy and weighted F1 0.8783. Accuracy and F1 0.8557 were the resultant scores of the binary version of the task, indicating that the model is a reliable variable in determining whether a sentence describes a financial event. Test pairs on 1,966 MATRES test pairs achieved 70.75 percent accuracy and weighted F1 0.6643. BEFORE and AFTER classification was reasonable at F1 0.7832 and 0.7125 respectively but VAGUE was only able to reach F1 0.0308 due to the severe imbalance caused by the classes and lack of full sentence context in MATRES. The event linking pipeline produced 1,358 confirmed pairs out of 3,000 FinRED sentences that were used to create a graph of 1,318 event nodes and 1,358 directed edges. The parallel pipeline showed a maximum speedup of 1.47x at 500 documents with the small gains being a reflection of the lightweight nature of the CPU-bound stages being parallelized.

There are a number of limitations of the system. The greatest one is that SIMULTANEOUS is not treated as a class of temporal relations. MATRES does not release usable SIMULTANEOUS labels with its release, and the class was

explicitly left out during training. This is an actual gap since many financial events can be simultaneous in nature in that two market indices are moving together or an announcement of earnings is coinciding with a trading halt. Future research ought to source or create SIMULTANEOUS-labeled financial data, possibly with a silver-label generation strategy, using sentences with two events co-occurring within the same clause. Future directions include providing full source sentences of MATRES event pairs instead of just trigger lemmas, using TB Dense as additional training data, using focal loss to address the imbalance of VAGUE classes and bringing FinBERT GPU inference to the parallel execution plan to achieve more substantial PDC speedups on larger document batches.

## ACKNOWLEDGMENT

The authors give credit to the original developers of the datasets and baseline systems that were employed in this work. We are grateful to Malo et al. with Financial PhraseBank, Ning et al. with MATRES and Sharma et al. with FinRED. We also credit the work of Huang et al., Ning et al., and Zhao et al. whose published systems were the baselines used in our experiments in temporal relation.

## REFERENCES

- [1] Q. Huang, Y. Hu, S. Zhu, Y. Feng, C. Liu, and D. Zhao, "More than classification: A Unified framework for event temporal relation extraction," in *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Toronto, Canada: Association for Computational Linguistics, 2023, pp. 9631–9646. [Online]. Available: <https://aclanthology.org/2023.acl-long.536/>
- [2] W. Ning, L. Li, X. Qin, Y. Feng, and J. Tang, "Temporal cognitive tree: A Hierarchical Modeling approach for event temporal relation extraction," in *Findings of the Association for Computational Linguistics: EMNLP 2024*. Miami, Florida, USA: Association for Computational Linguistics, 2024, pp. 834–849. [Online]. Available: <https://aclanthology.org/2024.findings-emnlp.47/>
- [3] J. Zhao, W. Ning, Y. Fei, Y. Feng, and L. Li, "GDLLM: A Global distance-aware modeling approach based on large language models for event temporal relation extraction," in *Findings of the Association for Computational Linguistics: EMNLP 2025*. Suzhou, China: Association for Computational Linguistics, 2025, pp. 8080–8091. [Online]. Available: <https://aclanthology.org/2025.findings-emnlp.428/>
- [4] T. Cassidy, B. McDowell, N. Chambers, and S. Bethard, "An annotation framework for dense event ordering," in *Proceedings of the 52nd Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*. Baltimore, Maryland: Association for Computational Linguistics, 2014, pp. 501–506. [Online]. Available: <https://aclanthology.org/P14-2082/>
- [5] Q. Zhang, "MATRES temporal relation resources (public repository)," <https://github.com/qiushizhang/MATRES>, 2021, accessed for MATRES pairs and evaluation protocol.
- [6] Q. Ning, H. Wu, and D. Roth, "A Multi-Axis Annotation Scheme for Event Temporal Relations," in *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Melbourne, Australia: Association for Computational Linguistics, 2018, pp. 4208–4217. [Online]. Available: <https://aclanthology.org/P18-1121/>
- [7] P. Malo, A. Sinha, P. Takala, P. Korhonen, and J. Järvinen, "Good Debt or Bad Debt: Detecting semantic orientations in economic texts," *Journal of the Association for Information Science and Technology*, vol. 65, no. 5, pp. 782–794, 2014. [Online]. Available: <https://doi.org/10.1002/asi.23162>
- [8] "ProsusAI/finbert (Hugging Face model hub)," <https://huggingface.co/ProsusAI/finbert>, 2019.
- [9] D. Araci, "FinBERT: Financial sentiment analysis with pre-trained language models," *arXiv preprint arXiv:2006.08097*, 2019. [Online]. Available: <https://arxiv.org/abs/2006.08097>

- [10] S. Sharma, T. Nayak, A. Bose, A. K. Meena, K. Dasgupta, N. Ganguly, and P. Goyal, "FinRED: A dataset for relation extraction in financial domain," in *Companion Proceedings of the Web Conference 2022 (WWW '22 Companion)*. ACM, 2022, pp. 234–239. [Online]. Available: <https://dl.acm.org/doi/10.1145/3487553.3524637>
- [11] Y. Liu, M. Ott, N. Goyal, J. Du, M. Joshi, D. Chen, O. Levy, M. Lewis, L. Zettlemoyer, and V. Stoyanov, "RoBERTa: A Robustly Optimized BERT Pretraining Approach," *arXiv preprint arXiv:1907.11692*, 2019. [Online]. Available: <https://arxiv.org/abs/1907.11692>